

Optimizarea interogarilor bazelor de date relationale (partea 1)

Ioana Ciuciu

ioana.ciuciu@ubbcluj.ro

<http://www.cs.ubbcluj.ro/~oana/>

Planificare

Sunt posibile mici
modificari ale planificarii in
timpul semestrului

Saptamana	Curs	Seminar	Laborator
S1	1. Concepte fundamentale ale bazelor de date. Modelare conceptuala	1. Modelul Entitate-Relatie. Modelul relational	1. Modelarea unei BD in modelul ER si implementarea ei in SQL Server
S2	2. Modelul relational de organizare a bazelor de date. Modelare conceptuala		
S3	3. Gestiunea bazelor de date relationale cu limbajul SQL (DDL)	2. Limbajul SQL – definirea si actualizarea datelor	2. Interogari SQL
S4	4. Gestiunea bazelor de date relationale cu limbajul SQL (DML)		
S5-6	5-6. Dependente functionale, forme normale	3. Limbajul SQL – regasirea datelor	3. Interogari SQL avansate
S7	7. JDBC (Java Database Connectivity)	4. Proceduri stocate	4. Proceduri stocate. View. Trigger
S8	8. Interogarea bazelor de date relationale cu operatori din algebra relationala		
S9	9. Structura fizica a bazelor de date relationale	5. View-uri. Functii definite de utilizator. Trigger	
S10-11	10-11. Indeksi. Arbori B. Fisiere cu acces direct	6. Formele normale ale unei relatii. Indeksi	
S12	12. Evaluarea interogarilor in bazele de date relationale		
S13	13. Extensii ale modelului relational si baze de date NoSQL	7. Probleme	Examen practic
S14	14. Aplicatii		

Partea 1

- ▶ Executia instructiunilor SQL
- ▶ Evaluarea operatorilor relationali



Executia instructiunilor SQL

- ▶ In aplicatii (la client) se cere executia unei instructiuni SQL
- ▶ **Caracteristici:**
 - ▶ se doreste un **timp de raspuns minim**, la orice interogare
 - ▶ la crearea instructiunii SQL nu e necesar sa se cunoasca structura fizica a bazei de date



Executia instructiunilor SQL

▶ **Etape in executia instructiunilor:**

- ▶ **la client:** instructiunea SQL (limbaj neprocedural) este generata si trimisa la server
- ▶ **la server** (unele dintre etape se pot parcurge in paralel):
 - ▶ se face **analiza** instructiunii SQL (analiza sintactica, semantica)
 - ▶ se translateaza intr-o **forma interna** (expresie in algebra relationala)
 - ▶ forma interna se transforma intr-o **forma optimala**
 - ▶ se genereaza un **plan de executie** (procedural)
 - ▶ se **evalueaza** planul generat si se trimite rezultatul la client



Executia instructiunilor SQL

- ▶ **Planurile de executie** generate se analizeaza:
 - ▶ periodic
 - ▶ dupa schimbarea structurii fizice
 - ▶ dupa multe operatii de modificare a tabelelor
- ▶ Din aceste analize se pot deduce **solutii de optimizare**, deci de modificare a structurii fizice
- ▶ La generarea planului de executie se folosesc **operatorii din algebra relationala**



Executia instructiunilor SQL

► Pentru **interogare** sunt necesari urmatoarii operatori:

- **Selectia:** $\sigma_c(R)$
- **Proiectia:** $\Pi_\alpha(R)$
- **Produsul cartezian:** $R1 \times R2$
- **Reuniunea:** $R_1 \cup R_2$
- **Diferenta:** $R_1 - R_2$
- **Intersectia:** $R_1 \cap R_2$
- **Joinul conditional:** $R_1 \otimes_\theta R_2$
- **Joinul natural:** $R_1 * R_2$
- **Joinul extern stanga:** $R1 \bowtie_c R2$
- **Joinul extern dreapta:** $R1 \bowtie_c R2$
- **Joinul extern complet:** $R1 \bowtie_c R2$
- **Semi joinul stanga:** $R1 \rhd R2$
- **Semi joinul dreapta:** $R1 \lhd R2$
- **Câtul:** $R1 \div R2$
- **Eliminarea duplicarilor:** $\partial(R)$
- **Sortarea inregistrarilor:** $S_{\{lista\}}(R)$
- **Gruparea:** $G_{\{lista1\} \text{ group by } \{lista2\}}(R)$



Executia instructiunilor SQL

- ▶ In SQL, o interogare poate fi scrisa in mai multe moduri
- ▶ **Exemplu** pentru o bdr (cheile primare sunt subliniate, cheile externe sunt marcate cu albastru):
 - ▶ `studenti(cnp, nume, prenume, grupa, media, adresa, email)`
 - ▶ `sectii(cod, denumire, act_functionare)`
 - ▶ `grupe(cod, sectia, anstudiu)`
- ▶ **Interogare.** Se cer studentii (nume, prenume, anstudiu, denumire sectie, media) de la o sectie data (ex. cu codul 2, poate fi parametru), cu media cel putin 9 (poate fi parametru). Inregistrare se cer ordonate alfabetic pe sectii si ani de studiu.



Executia instructiunilor SQL

- ▶

```
select nume,prenume, anstudiu, denumire, media
    from studenti st, grupe gr, sectii se
   where st.grupa=gr.cod and gr.sectia=se.cod
   and sectia=2 and media>=9
   order by anstudiu, nume, prenume
```
- ▶

```
select nume,prenume, anstudiu, denumire, media
    from (studenti st inner join grupe gr on st.grupa=gr.cod)
   inner join sectii se on gr.sectia=se.cod
   where sectia=2 and media>=9
   order by anstudiu, nume, prenume
```
- ▶

```
select nume, prenume, anstudiu, denumire, media
    from
      ((select nume, prenume, grupa, media from studenti where media>=9)
    st inner join
      (select * from grupe where sectia=2) gr on st.grupa=gr.cod
    ) inner join (select * from sectii where sectia=2) se
    on gr.sectia=se.cod
   order by anstudiu, nume, prenume
```



Executia instructiunilor SQL

- ▶ Toate aceste variante sunt echivalente (furnizeaza acelasi rezultat)
- ▶ Expresiile echivalente în algebra relationala sunt urmatoarele:

$$\Pi_{\beta}(s_{\alpha}(\sigma_C(\text{studenti} \times \text{grupe} \times \text{sectii})))$$

$$\Pi_{\beta}(s_{\alpha}(\sigma_{C1}((\text{studenti} \otimes_{C2} \text{grupe}) \otimes_{C3} \text{sectii})))$$

$$\Pi_{\beta}(s_{\alpha}(\sigma_{C1}((\sigma_{C2}(\text{studenti}) \otimes_{C3}(\sigma_{C4}(\text{grupe}))) \otimes_{C5}(\sigma_{C6}(\text{sectii}))))))$$



Executia instructiunilor SQL

- ▶ **Pentru o expresie in algebra relationala se poate construi un arbore de evaluare**
- ▶ **Probleme:**
 - ▶ care varianta este mai avantajoasa?
 - ▶ ce parametri se optimizeaza la generarea planului de executie?
 - ▶ la generarea planului de executie, ce informatii sunt necesare?
 - ▶ ce poate face un optimizator (componenta a SGBD)?



Evaluarea operatorilor relationali

- ▶ Operanzii pentru operatorii relationali sunt:
 - ▶ **tabele din baza de date** - pot sa existe **indecsi** atasati
 - ▶ **viewuri sau tabele temporare** (obtinute prin evaluarea unor operatori relationali) – pentru care nu exista indecsi
- ▶ **Pentru un operator din algebra relationala pot exista mai multi algoritmi de evaluare**
- ▶ La generarea planului de executie se alege operatorul cu complexitatea cea mai mica in contextul existent al bazei de date
 - ▶ se iau informatii din dictionarul bazei de date si informatii statistice preluate intr-o perioada de timp de la executia diverselor instructiuni



Evaluarea operatorilor relationali - Algoritmi

AI. Table Scan

- ▶ Pentru multi operatori este necesara parcurgerea tuturor inregistrarilor dintr-un tabel (in ordinea furnizata de unele tabele de alocare, de un clustered index sau un nonclustered index)
- ▶ Fie **bR** numarul de blocuri in care se memoreaza inregistrarile tabelului
 - ▶ Din algoritmul de cautare secventiala se deduce ca aproximativ **bR/2** blocuri sunt necesare (in medie) la o cautare secventiala dupa valorile unei chei
 - ▶ **Toate** blocurile care contin inregistrari trebuie transferate în memoria interna (la o cautare secventiala dupa valorile unui atribut, selectie dupa o conditie complexa)
 - ▶ Timpul de transfer este mare pentru tabele cu multe inregistrari



Evaluarea operatorilor relationali - Algoritmi

A2. Index Seek

- ▶ Acest algoritm este cel mai performant si se foloseste la **cautarea dupa valoarea unei chei C**
- ▶ Cautarea este de forma: **$C=K0$**
- ▶ Aceasta cautare este **explicit** precizata (cautare in tabel, evaluare join) sau **implicita** (verificarea restrictiei de cheie)
- ▶ Se consulta un index (construit implicit pentru **restrictia de cheie unica** sau construit prin instructiunea **create index**) memorat ca un **B-arbore** sau **B+-arbore**.
- ▶ **Observatie.** **C** poate fi o cheie simpla, compusa, sau o expresie



Evaluarea operatorilor relationali - Algoritmi

A3. Index Scan

- ▶ Acest algoritm se foloseste pentru evaluarea operatorului $\sigma_c(R)$, unde conditia **C** este de forma:
- ▶ **$A < v, A \leq v, A > v, A \geq v, A \text{ is null}, A \text{ is not null}$** , indexul s-a construit pentru o **cheie A**
- ▶ **$A = v, A < v, A \leq v, A > v, A \geq v, A \text{ is null}, A \text{ is not null}$** , indexul s-a construit pentru un **atribut** sau o **expresie A**
- ▶ Inregistrarile se obtin prin parcurgerea partiala sau totala a indexului si citirea inregistrarilor din relatie conform adreselor furnizate de index (unele blocuri se pot citi de mai multe ori)



Evaluarea operatorilor relationali - Algoritmi

A3. Index Scan

► Observatii.

- Timpul necesar pentru executia acestui algoritm poate fi mare si e posibil ca algoritmul **table scan** sa fie mai avantajos. Daca se poate estima numarul de inregistrari furnizate, atunci se poate deduce care algoritm ar fi mai eficient. Aceasta estimare se poate face daca se determina numarul de inregistrari obtinute de aceeasi cautare la evaluari anterioare (intr-un "istoric al evaluarilor")
- Daca o astfel de estimare nu este posibila, atunci prin sirul de caractere din instructiunea select-SQL se **poate preciza** varianta propusa de client
- Pentru o conditie de forma "**A<>v**" se poate folosi un algoritm **table scan** sau sa se foloseasca de doua ori algoritmul index scan (**A<>v** $\equiv$ (**A<v**) or (**A>v**))



Evaluarea operatorilor relationali - Algoritmi

A3. Index Scan

► Observatii.

- Pentru o conditie de forma "**(A=v) and C**", unde pentru **A** s-a construit un index, iar **C** este de una din formele amintite la **index scan**, avem urmatoarea evaluare:

$$\sigma_C(R) = \sigma_{C1}(\sigma_{A=v}(R))$$

- Pentru selectia interioara se foloseste un **index seek** sau **index scan**, iar pentru selectia exterioara un **index scan**
- Scrierea conditiei **C** in aceasta forma este utila daca sistemul **nu poate** sau **nu face** o transformare a conditiei (din analiza planului de executie se observa daca sistemul transforma conditia in aceasta varianta)
- Se poate descrie un algoritm pentru evaluarea operatorului de selectie prin sortarea sursei de date



Evaluarea operatorilor relationali - Algoritmi

A4. Cross Join

- ▶ Acest algoritm se foloseste la evaluarea unui **produs cartezian**:
 - ▶ **R cross join S**
 - ▶ **R inner join S on true**
 - ▶ **select ... from R, S ...**, iar intre R si S nu exista conditie de legatura



Evaluarea operatorilor relationali - Algoritmi

A4. Cross Join

► Fie:

- **b_R, b_S** - numarul de blocuri in care se memoreaza **R** , respectiv **S**
- **m, n** - numarul de blocuri ce pot apare simultan in memoria interna din **R** , respectiv **S** (in memoria interna exista $m+n$ zone tampon pentru cele doua tabele)



Evaluarea operatorilor relationali - Algoritmi

A4. Cross Join

- ▶ Pentru a genera produsul cartezian $\{(r, s) \mid r \in R, s \in S\}$ se poate folosi urmatorul algoritm:
 - ▶ pentru fiecare grup de maxim **m** blocuri din **R**:
 - ▶ se citeste grupul de blocuri din **R** in memoria interna. Fie **M1** multimea de inregistrari din aceste blocuri
 - ▶ pentru fiecare grup de maxim **n** blocuri din **S**:
 - ▶ se citeste grupul de blocuri din **S** in memoria interna. Fie **M2** multimea de inregistrari din aceste blocuri
 - ▶ pentru fiecare **r** $\in$ **M1**:
 - pentru fiecare **s** $\in$ **M2** se include **(r, s)** in rezultat



Evaluarea operatorilor relationali - Algoritmi

A4. Cross Join

- ▶ Complexitatea algoritmului: **numarul total de blocuri citite** (din cele doua tabele):

$$b_R + \left\lceil \frac{b_R}{m} \right\rceil * b_S$$

(numarul de blocuri citite din R, pentru fiecare grup de maxim **m** blocuri din R se citeste S)

- ▶ Pentru a *minimiza* aceasta valoare ar trebui sa alegem pe **m** *maxim* posibil (restul operanzilor sunt constante)
- ▶ In memoria interna se poate alege o singura zona tampon pentru S (deci $n=1$) si spatiul ramas sa se foloseasca la maxim pentru R (m maxim)
- ▶ Daca se face un schimb intre cele doua relatii (in algoritm si in calculul complexitatii) se obtine o complexitate:

$$b_S + \left\lceil \frac{b_S}{n} \right\rceil * b_R$$



Evaluarea operatorilor relationali - Algoritmi

A4. Cross Join

- ▶ Dintre cele doua variante de lucru este avantajoasa varianta care da complexitatea cea mai mica
- ▶ Daca dimensiunea zonelor tampon este aceeași pentru R și S, atunci este optima situatia in care ca **prim tabel** se ia acela cu *dimensiune mai mica* ($\min\{bR, bS\}$)
- ▶ **Observatie.** Daca una din sursele de date incapă in memoria interna, atunci complexitatea algoritmului este **$bR + bS$**



Evaluarea operatorilor relationali - Algoritmi

A5. Nested Loops Join

- ▶ Algoritmul Cross Join se poate folosi si la evaluarea unui join (intern, extern, semijoin) intre doua tabele daca se foloseste o **conditie compusa pentru join**
- ▶ La fiecare element **(r, s)** din produsul cartezian se evalueaza si conditia din operatorul de join



Evaluarea operatorilor relationali - Algoritmi

A6. Indexed Nested Loops Join

- ▶ Acest algoritm se foloseste pentru evaluarea $R \bowtie_{\theta} S$, unde $C \equiv (R.A = S.B)$, iar pentru A (din R) sau B (din S) **exista un index**
- ▶ In descrierea algoritmului presupunem ca pentru B din tabelul S exista un index



Evaluarea operatorilor relationali - Algoritmi

A6. Indexed Nested Loops Join

pentru fiecare bloc din R:

- citește blocul din R în memoria internă. Fie M multimea de înregistrări citite
- pentru fiecare $\mathbf{r} \in M$:
 - determină $v = \mathbf{A}(\mathbf{r})$
 - folosind indexul pentru B din S se determină înregistrările $\mathbf{s} \in S$ cu valoarea v pentru B (în funcție de tipul de index, pot fi mai multe înregistrări). Pentru fiecare înregistrare $\mathbf{s}$ determinată, perechea $(\mathbf{r}, \mathbf{s})$ se adaugă la rezultat



Evaluarea operatorilor relationali - Algoritmi

A7. Merge Join

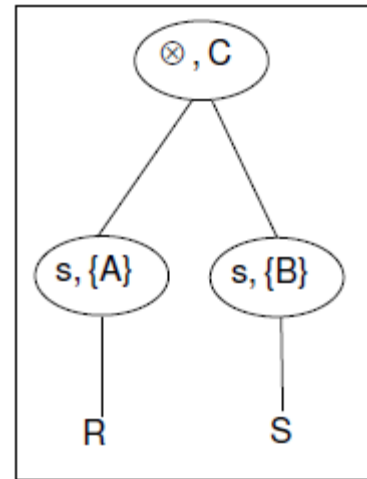
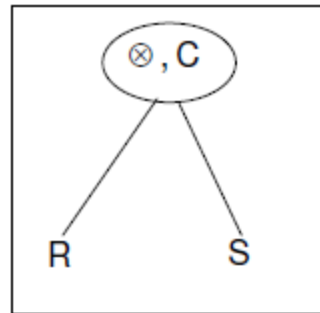
- ▶ Acest algoritm se foloseste pentru evaluarea $R \bowtie_{\theta} S$, unde $C \equiv (R.A=S.B)$, iar pentru A (din R) sau B (din S) **nu exista index**
- se sorteaza cele doua tabele dupa coloanele folosite la join: **R dupa A si S dupa B**
- se parcurg în paralel cele doua tabele obtinute. Fie r din A si s din B doua înregistrari curente
 - daca $r.A=s.B$, atunci (r',s') se adauga la rezultat; r' este in multimea inregistrarilor consecutive din R cu valoarea pentru A egala cu r.A, s' analog pentru S; next(r); next(s) (se ia cate o înregistrare cu urmatoarea valoare pentru A si S)
 - daca $r.A < s.B$, atunci next(r) (se determina o înregistrare din R sortat cu urmatoarea valoare pentru A)
 - daca $r.A > s.B$, atunci next(s) (se determina o înregistrare din S sortat cu urmatoarea valoare pentru B)



Evaluarea operatorilor relationali - Algoritmi

A7. Merge Join

- ▶ Acest algoritm inlocuieste un arbore de evaluare cu alt arbore



Evaluarea operatorilor relationali - Algoritmi

A8. Hybrid Merge Join

- ▶ Acest algoritm se foloseste pentru evaluarea $R \bowtie_{\theta} S$, unde $C \equiv (R.A=S.B)$, iar pentru unul dintre attributele: A (din R) sau B (din S) este sortata sursa sau se sorteaza, iar pentru celalalt atribut exista un index de tip B+-arbore
- ▶ In acest caz se face o interclasare intre sursa sortata si terminalele aflate în indexul B+-arbore construit pentru al doilea tabel



Evaluarea operatorilor relationali - Algoritmi

A9. Hash Join

- ▶ Acest algoritm se foloseste pentru evaluarea $R \bowtie_{\theta} S$, unde $C \equiv (R.A = S.B)$
- 1. Se alege unul dintre tabele (se recomanda cel cu dimensiune mai mica) si se memoreaza (în memoria interna, sau în fisiere temporare) pe blocuri, conform unei functii de dispersare **h**. Presupunem ca acest tabel este S
- 2. Pentru fiecare $r \in R$ se determina $v = h(r.A)$. Inregistrarea r se compara numai cu inregistrarile din S care sunt in blocul $h(v)$



Evaluarea operatorilor relationali - Algoritmi

A9. Hash Join

► Observatii.

- Eficienta este mare daca unul dintre tabele se poate memora in memoria interna
- Rezultate bune se obtin si pentru tabele cu dimensiune mare.
- In cazul in care conditia de join este de forma:

$$\mathbf{C = (R.A1 = S.B1) [and (R.A2 = S.B2)] ...}$$

si pentru una din coloanele A1, A2, ..., B1, B2, ... exista un index, atunci se poate folosi unul din algoritmii **Merge Join** sau **Hash Join** modificat

- In cazul în care conditia de join este complexa, se poate aduce la **forma normala conjunctiva** si sa se analizeze fiecare termen



Evaluarea operatorilor relationali - Algoritmi

Joinuri externe

- ▶ Se adapteaza algoritmi de la joinurile conditionale

Joinuri multiple

- ▶ La evaluarea unor joinuri multiple se pot folosi proprietatile de asociativitate si comutativitate
- ▶ Exemplu pentru joinul natural a patru tabele folosind numai asociativitatea
 - ▶ $((R1 * R2) * R3) * R4$
 - ▶ $(R1 * R2) * (R3 * R4)$
 - ▶ $R1 * (R2 * (R3 * R4))$
 - ▶ $(R1 * (R2 * R3)) * R4$
 - ▶ $R1 * ((R2 * R3) * R4)$



Evaluarea operatorilor relationali - Algoritmi

Joinuri multiple

- ▶ La fiecare expresie se poate asocia un arbore de evaluare, unde in nodurile interioare sunt rezultate intermediare
- ▶ Dintre planurile de executie posibile se foloseste acela cu un "timp de evaluare estimat" minim

Operatii pe multimii de inregistrari:

$$R_1 \cup R_2 \quad R_1 - R_2 \quad R_1 \cap R_2$$

- ▶ Se adapteaza algoritmii precedenti (cei de la join)



Partea 2

- ▶ Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare
- ▶ Forma optimala pentru expresia din algebra relationala



Algoritmi pentru evaluarea operatorilor relationali

- ▶ **A1. Table Scan**
- ▶ **A2. Index Seek**
- ▶ **A3. Index Scan**
- ▶ **A4. Cross Join**
- ▶ **A5. Nested Loops Join**
- ▶ **A6. Indexed Nested Loops Join**
- ▶ **A7. Merge Join**
- ▶ **A8. Hybrid Merge Join**
- ▶ **A9. Hash Join**
- ▶ **Joinuri externe**
- ▶ **Joinuri multiple**
- ▶ **Operatii pe mulimi de inregistrari: $R1 \cup R2, R1 - R2, R1 \cap R2$**



Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare

AI0. Sortarea

- ▶ Ceruta explicit:

```
select ... order by lista  
select distinct ...
```

- ▶ Necesara la evaluarea unor operatori (join, intersectie, reuniune, diferenta)

- ▶ Necesara la gruparea inregistrarilor:

```
select ... group by lista
```



Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare

- ▶ A10. Sortarea
- ▶ Varianta 1: sursa pentru sortare incapa in memoria interna, atunci se foloseste un algoritm de **sortare interna**
- ▶ Varianta 2: sursa pentru sortare nu incapa in memoria interna:
 - ▶ se creaza **monotonii** (inregistrari din sursa care incapa in memoria interna si se ordoneaza)
 - ▶ monotoniile rezultate se distribuie in fisiere temporare
 - ▶ cu un algoritm de **sortare externa** se face o interclasare a monotoniilor



Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare

A10. Sortarea

- ▶ **Precizari.** In exemplele urmatoare (la descrierea algoritmilor de sortare externa):
 1. se foloseste un tabel cu 3100 înregistrari
 2. o monotonie poate contine cel mult 100 înregistrari
 3. pentru distribuirea monotoniilor se folosesc patru fisiere
 4. cu x^y s-a notat repetarea de y ori a unei monotonii de lungime relativa x (unde monotonia de lungime relativa 1 este cea rezultata printr-un algoritm de sortare interna)
- ▶ In continuare se vor descrie urmatorii algoritmi de sortare externa:
 - ▶ Interclasarea echilibrata
 - ▶ Interclasarea polifazica
 - ▶ Interclasarea in cascada



Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare

AI0. Sortarea

▶ I. Interclasarea echilibrata

- ▶ monotonii initiale se distribuie relativ uniform pe jumatate din fisiere
- ▶ prin interclasari se creaza monotonii care se distribuie pe fisierele ramase
 - ▶ aceasta etapa se repeta pana la obtinerea unei singure monotonii

Exemplu:

F1	F2	F3	F4	Observații
1 ¹⁶	1 ¹⁵	-	-	distribuirea monotoniiilor
-	-	2 ⁸	2 ⁷ 1 ¹	interclasare, alternativ pe F3, F4; copierea unei monotonii
4 ⁴	4 ³ 3 ¹	-	-	interclasare, alternativ pe F1, F2
-	-	8 ²	8 ¹ 7 ¹	interclasare, alternativ pe F3, F4
16 ¹	15 ¹	-	-	interclasare, alternativ pe F1, F2
-	-	31 ¹	-	interclasare, obținere rezultat final în F3

Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare

AI0. Sortarea

► 2. Interclasarea polifazica

- Monotoniile se distribuie in fisiere, dupa o anumita regula (trebuie determinata o configuratie initiala), iar dupa distribuire ramane liber un singur fisier
- In fisierul liber se creaza monotonii prin interclasare din toate celelalte fisiere pana la eliberarea unui fisier
 - aceasta etapa se repeta pana la obtinerea unei singure monotonii

Exemplu:

F1	F2	F3	F4	Observații
1 ¹³	1 ¹¹	1 ⁷	-	distribuirea monotoniilor
1 ⁶	1 ⁴	-	3 ⁷	interclasare în F4 până la golire F3
1 ²	-	5 ⁴	3 ³	interclasare în F3 până la golire F2
-	9 ²	5 ²	3 ¹	interclasare în F2 până la golire F1
17 ¹	9 ¹	5 ¹	-	interclasare în F1 până la golire F4
-	-	31 ¹	-	interclasare, obținere rezultat final în F4

Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare

Probleme:

a. Determinarea **configuratiei initiale** (ca numar de monotonii)

F1	F2	F3	F4	
1	-	-	-	monotonia din F1 se obține din celelalte fișiere
-	1	1	1	monotonia din F4 se obține din celelalte fișiere
1	2	2	-	monotoniile din F3 se obțin din celelalte fișiere
3	4	-	2	monotoniile din F2 se obțin din celelalte fișiere
7	-	4	6	monotoniile din F1 se obțin din celelalte fișiere
-	7	11	13	monotoniile din F4 se obțin din celelalte fișiere
13	20	24	-	monotoniile din F3 se obțin din celelalte fișiere
c_n	b_n	a_n	-	$a_n \geq b_n \geq c_n$
$c_n + a_n$	$b_n + a_n$	-	a_n	

- **Procedeu** se poate folosi pentru orice numar de fisiere

b. Rezolvarea cazului in care nu exista suficiente monotonii pentru crearea unei "**distributii initiale perfecte**": folosirea unor "monotonii vide", care se interclaseaza cu monotonii reale

Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare

AI0. Sortarea

▶ 3. Interclasarea in cascada

- ▶ Monotoniile se distribuie in fisiere, dupa o anumita regula (este necesara o configuratie initiala pentru distribuirea monotoniilor), iar în final ramane liber un singur fisier
- ▶ In fisierul ramas liber se creeaza monotonii prin interclasare pana la eliberarea unui fisier. Se continua interclasarea in fisierul eliberat cu monotonii din restul de fisiere. Aceasta etapa se repeta pana raman monotonii intr-un singur fisier, care se copiaza in fisierul liber
- ▶ Etapa precedenta se repeta pana la obtinerea unei singure monotonii finale

F1	F2	F3	F4	Observații
1 ¹⁴	1 ¹¹	1 ⁶	-	distribuirea monotoniilor
-	1 ³	2 ⁵	3 ⁶	interclasare: (F1,F2,F3) în F4, (F1,F2) în F3; copiere F1 în F2
6 ³	5 ²	3 ¹	-	interclasare: (F2,F3,F4) în F1, (F3,F4) în F2; copiere F4 în F3
-	6 ¹	11 ¹	14 ¹	interclasare: (F1,F2,F3) în F4, (F1,F2) în F3; copiere F1 în F2
31 ¹	-	-	-	interclasare (F2,F3,F4) în F1 rezultat final în F1

Algoritmi pentru evaluarea operatorilor relationali: Algoritmi de sortare

Probleme:

- a. Determinarea configuratiei initiale

F1	F2	F3	F4
1	-	-	-
-	1	1	1
1	1	1	
1	1		
1			
3	2	1	-
	3	3	3
		2	2
			1
-	3	5	6
6	6	6	
5	5		
3			
14	11	6	-
	14	14	14
		11	11
			6
-	14	25	31

c_n	b_n	a_n	-	$a_n \geq b_n \geq c_n$
$c_n + b_n + a_n$	$b_n + a_n$	-	a_n	

- **Observatie.** Procedul se poate folosi pentru orice numar de fisiere
- b. Rezolvarea cazului in care nu exista suficiente monotonii pentru o "distributie perfecta"

Forma optimala pentru expresia din algebra relationala

- ▶ **Instructiunea SQL** se transforma intr-o **expresie din algebra relationala** (se poate genera usor, conform unor reguli de transformare pentru fiecare clauza din instructiunea SQL)
- ▶ **Fara informatii** din baza de date, **expresia relationala se poate transforma la o forma optimala** (algoritmul de evaluare are o complexitate mai mica)
- ▶ Se folosesc anumite reguli de transformare (proprietati matematice ale operatorilor relationali)
- ▶ Fiecare SGBD foloseste anumite reguli de transformare
- ▶ La scrierea (generarea) unei instructiuni SQL trebuie luate in considerare si optimizarile pe care le face serverul de date



Forma optimala pentru expresia din algebra relationala

- ▶ **RI.** $\sigma_C(\Pi_\alpha(R)) = \Pi_\alpha(\sigma_C(R))$
 - ▶ selectia reduce numarul de inregistrari pentru proiectie, deci proiectia in varianta din dreapta va analiza mai putine inregistrari
 - ▶ se poate crea un algoritm care evalueaza cei doi operatori printr-o singura parcurgere a relatiei R (optimizare in unele SGBD)



Forma optimala pentru expresia din algebra relationala

- ▶ **R2**. In loc de doua parcurgeri se face una singura:

$$\sigma_{C1}(\sigma_{C2}(R)) = \sigma_{C1-and-C2}(R)$$

$$\Pi_{\alpha}(\Pi_{\beta}(R)) = \Pi_{\alpha \cap \beta}(R)$$

- ▶ **R3**. Inlocuire produs cartezian si selectie cu join conditional (pentru joinul conditional exista algoritmi care nu evalueaza produsul cartezian)

$$\sigma_C(R \times S) = R \otimes_C S$$

unde C este conditia de legatura dintre R si S



Forma optimala pentru expresia din algebra relationala

- ▶ **R4**. Folosirea unor proprietati de distributivitate (a unui operator unar "**f**" relativ cu un operator binar " $\odot$ "):

$$f(a \odot b) = f(a) \odot f(b)$$

- ▶ **R4.a**. σ este distributiv fata de $\cup, \cap, -$ (sursele R si S trebuie sa aiba scheme compatibile)

$$\sigma_C(R \cup S) = \sigma_C(R) \cup \sigma_C(S)$$

$$\sigma_C(R \cap S) = \sigma_C(R) \cap \sigma_C(S)$$

$$\sigma_C(R - S) = \sigma_C(R) - \sigma_C(S)$$

- ▶ **R4.b**. σ Este distributiv fata de "**x**":

$$\sigma_C(R \times S) = \sigma_C(R) \times \sigma_C(S)$$



Forma optimala pentru expresia din algebra relationala

Cazuri particulare (R4):

- ▶ Daca in C se folosesc numai attribute din R : $\sigma_C(R \times S) = \sigma_C(R) \times S$
- ▶ In cazul in care **$C = C1 \text{ and } C2$** , iar in $C1$ se folosesc numai attribute din R si în $C2$ se folosesc numai attribute din S :

$$\sigma_{C1-and-C2}(R \times S) = \sigma_{C1}(R) \times \sigma_{C2}(S)$$

- ▶ In cazul in care **$C = C1 \text{ and } C2$** , unde **$C2$** este o conditie de legatura intre R si S :

$$\sigma_{C1-and-C2}(R \times S) = \sigma_{C1}(R \otimes_{C2} S)$$



Forma optimala pentru expresia din algebra relationala

- ▶ **R4**. Folosirea unor proprietati de distributivitate (a unui operator unar "**f**" relativ cu un operator binar " $\odot$ "):

$$f(a \odot b) = f(a) \odot f(b)$$

- ▶ **R4.c**. Π este distributiv fata de $\cup$, $\cap$:

$$\Pi_{\alpha}(R \cup S) = \Pi_{\alpha}(R) \cup \Pi_{\alpha}(S)$$

$$\Pi_{\alpha}(R \cap S) = \Pi_{\alpha}(R) \cap \Pi_{\alpha}(S)$$

- ▶ **Observatie.** Π nu este distributiv fata de "-"

- ▶ **Exemplu:** $\Pi_{\{x\}} \left(\begin{matrix} x & y \\ \left[\begin{matrix} 1 & 2 \\ 1 & 3 \\ 2 & 3 \end{matrix} \right] - \left\{ \begin{matrix} x & y \\ 2 & 3 \\ 1 & 4 \end{matrix} \right\} \end{matrix} \right)$



Forma optimala pentru expresia din algebra relationala

- ▶ **R4**. Folosirea unor proprietati de distributivitate (a unui operator unar "**f**" relativ cu un operator binar " $\odot$ "):

$$f(a \odot b) = f(a) \odot f(b)$$

- ▶ **R4.d**. Π si σ (sortarea) sunt distributive fata de "join" (fata de orice operator de join)

- ▶ Pentru optimizare se mai poate face o transformare

$$\Pi_{\alpha}(R \text{ -- join -- } S) = \Pi_{\alpha}(\Pi_{\alpha_1}(R) \text{ join } \Pi_{\alpha_2}(S))$$

unde:

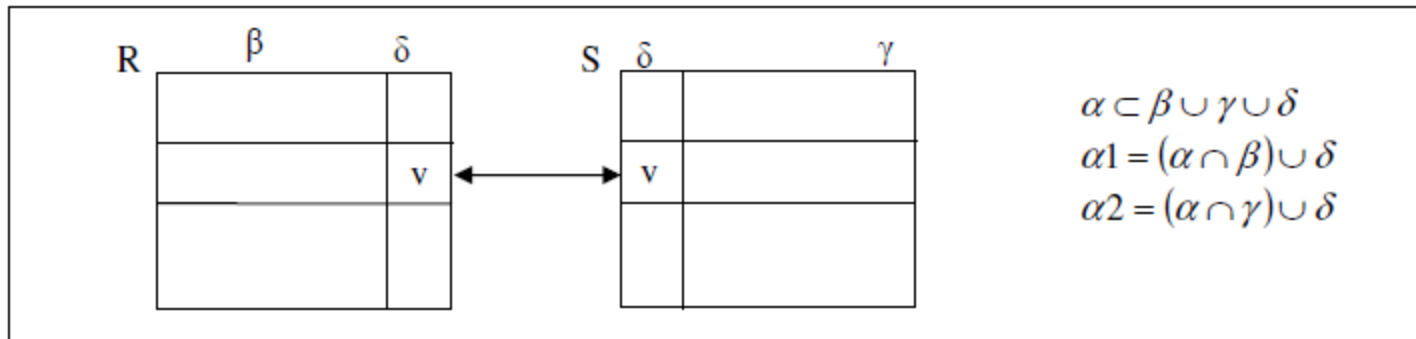
$$\alpha_1 = ((\text{attributele} - \text{din} - \alpha) \cap (\text{attributele} - \text{din} - R)) \cup (\text{attributele} - \text{pt} - \text{join})$$

$$\alpha_2 = ((\text{attributele} - \text{din} - \alpha) \cap (\text{attributele} - \text{din} - S)) \cup (\text{attributele} - \text{pt} - \text{join})$$



Forma optimala pentru expresia din algebra relationala

- **Obs.** La α_1 si α_2 se vor adauga attributele utile *operatorilor precedenti (in expresia relationala)*. Din sursele R si S se selecteaza numai attributele utile pentru join si cele dorite in final



Forma optimala pentru expresia din algebra relationala

- ▶ **Asociativitate si comutativitate** pentru unii operatori relationali
 - ▶ **R5. a.** Asociativitate si comutativitate pentru $\cup$ si $\cap$
 - ▶ **R5. b.** Asociativitate pentru produs cartezian si join natural
 - ▶ **R5. c.** Rezultate "echivalente" (aceleasi înregistrari, dar coloanele apar în alta ordine) la comutarea operanzilor pentru "x" si "join"

Obs. Este posibil ca sa nu se poata folosi asociativitatea pentru "join conditional" (depinde de conditiile de legatura)

$R \times S = S \times R$ cu algoritmul **Cross Join** este importanta ordinea surselor de date

$R \text{ join } S = S \text{ join } R$ cu algoritmul **Hash Join** este important ca functia hash sa fie folosita pentru sursa cu dimensiune mai mica



Forma optimala pentru expresia din algebra relationala

- ▶ **R6. Tranzitivitatea** unor operatori relationali pentru operatori de join - pot apare unele filtrari suplimentare inainte de join:

- ▶ $(A > B \text{ and } B > 3) \equiv (A > B \text{ and } B > 3 \text{ and } A > 3)$

A apare in R, B apare in S:

$$R \otimes_{A > B \text{ and } B > 3} S = (\sigma_{A > 3}(R)) \otimes_{A > B} (\sigma_{B > 3}(S))$$

- ▶ $(A = B \text{ and } B = 3) \equiv (A = B \text{ and } B = 3 \text{ and } A = 3)$

A apare in R, B apare in S:

$$R \otimes_{A = B \text{ and } B = 3} S = (\sigma_{A = 3}(R)) \otimes_{A = B} (\sigma_{B = 3}(S))$$



Forma optimala pentru expresia din algebra relationala

- ▶ **R7. Evaluare** $\sigma_C(R)$, unde $C \equiv (R.A \in \delta(\Pi_{\{B\}}(S)))$
- ▶ Pentru a nu evalua C pentru fiecare inregistrare din R , evaluarea initiala este echivalenta cu: $R \otimes_{R.A=S.B} (\delta(\Pi_{\{B\}}(S)))$



Forma optimala pentru expresia din algebra relationala

- ▶ **Exemplu** pentru urmatoarea bdr (cheile primare sunt subliniate, cheile externe sunt marcate cu albastru):
 - ▶ `studenti(cnp, nume, prenume, grupa, media,...)`
 - ▶ `sectii(cod, denumire,...)`
 - ▶ `grupe(cod, sectia, anstudiu,...)`
- ▶ **Interogare.** Se cer studentii (nume, prenume, anstudiu, denumire sectie, media) de la o sectie data (ex. cu codul 2, poate fi parametru), cu media cel putin 9 (poate fi parametru). Inregistrarile se cer ordonate alfabetic pe sectii si ani de studiu

```
select nume,prenume, anstudiu, denumire, media
from studenti, grupe, sectii
where studenti.grupa= grupe.cod and grupe.sectia= sectii.cod and sectia=2
and media>=9
order by anstudiu, nume, prenume
```



Forma optimala pentru expresia din algebra relationala

► Daca notam:

$C = (\text{studenti.grupa} = \text{grupe.cod} \text{ and } \text{grupe.sectia} = \text{sectii.cod} \text{ and } \text{sectia} = 2 \text{ and } \text{media} \geq 9)$

$\alpha = \{\text{anstudiu}, \text{nume}, \text{prenume}\}$ - **atributele de la sortare**

$\beta = \{\text{nume}, \text{prenume}, \text{anstudiu}, \text{denumire}, \text{media}\}$ - **atributele de la selectie**
atunci expresia relationala atasata interogarii este:

$$e = \Pi_{\beta} \left(S_{\alpha} \left(\sigma_C (\text{studenti} \times \text{grupe} \times \text{sectii}) \right) \right)$$



Forma optimala pentru expresia din algebra relationala

Conform regulilor precedente vom face unele transformări.

- asociativitate pentru "x":

`studenti x grupe x sectii = (studenti x grupe) x sectii`

sau:

`studenti x grupe x sectii = studenti x (grupe x sectii)`

- distributivitate σ față de "x", folosirea unui caz particular și folosirea tranzitivității operatorului de egalitate:

`(grupe.sectia = sectii.cod and sectia=2)`
`= (grupe.sectia = sectii.cod and sectia=2 and sectii.cod=2)`

<code>studenti.grupa = grupe.cod and grupe.sectia = sectii.cod and sectia=2 and media>=9 and sectii.cod=2</code>				
C1	C2	C3	C4	C5

$\sigma_C((\text{studenti} \times \text{grupe}) \times \text{sectii}) =$

$= \sigma_{C1 \text{ and } C2}((\sigma_{C4}(\text{studenti}) \times \sigma_{C3}(\text{grupe})) \times \sigma_{C5}(\text{sectii}))$

sau:

$= \sigma_{C1 \text{ and } C2}(\sigma_{C4}(\text{studenti}) \times (\sigma_{C3}(\text{grupe}) \times \sigma_{C5}(\text{sectii})))$



Forma optimala pentru expresia din algebra relationala

- înlocuire selecție și produs cartezian prin join condițional:

$$= ((\sigma_{C4}(\text{studenti})) \otimes_{C1} (\sigma_{C3}(\text{grupe}))) \otimes_{C2} \sigma_{C5}(\text{sectii})$$

sau:

$$= (\sigma_{C4}(\text{studenti})) \otimes_{C1} ((\sigma_{C3}(\text{grupe})) \otimes_{C2} (\sigma_{C5}(\text{sectii})))$$

Pe baza informațiilor statistice din baza de date se alege una dintre variante (vom lua în considerare prima variantă).

$$\Rightarrow e = \Pi_{\beta} (S_{\alpha} ((\sigma_{C4}(\text{studenti})) \otimes_{C1} (\sigma_{C3}(\text{grupe}))) \otimes_{C2} (\sigma_{C5}(\text{sectii})))$$

- distributivitate Π față de "join":

$\beta_1 = \{\text{nume, prenume, media, grupa}\}$ - utile pentru β și join

$\beta_2 = \{\text{cod, denumire}\}$ - utile pentru β și join

$\beta_3 = \{\text{cod, sectia, anstudiu}\}$ - utile pentru β și join

$$e = \Pi_{\beta} (S_{\alpha} ((\Pi_{\beta_1}(\sigma_{C4}(\text{studenti}))) \otimes_{C1} (\Pi_{\beta_2}(\sigma_{C3}(\text{grupe})))) \otimes_{C2} (\Pi_{\beta_3}(\sigma_{C5}(\text{sectii}))))$$



Forma optimala pentru expresia din algebra relationala

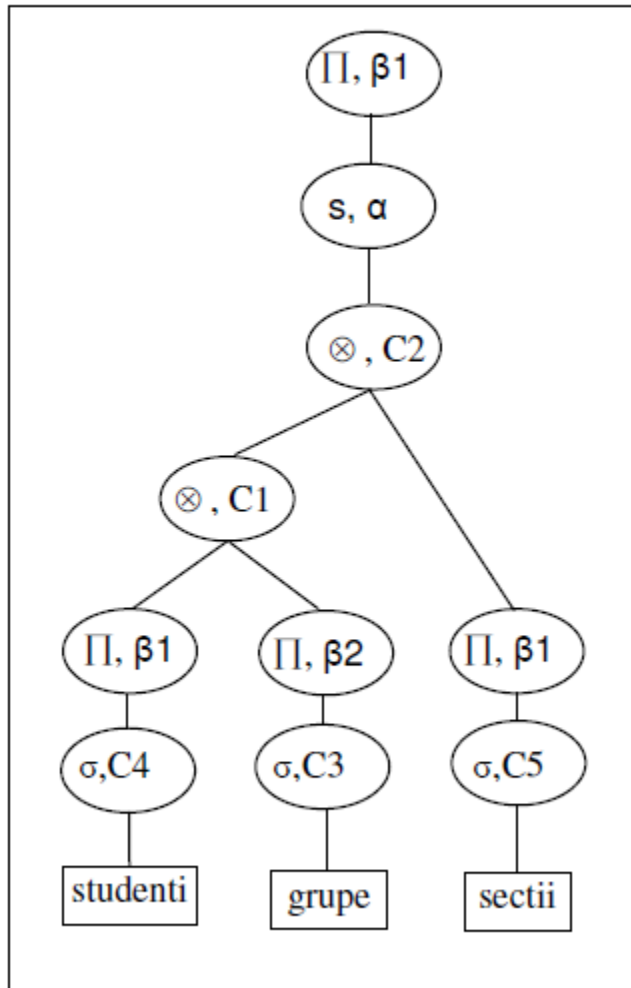
Ultima expresie corespunde la instrucțiunea:

```
select nume, prenume, anstudiu, denumire, media
from
  ((select nume, prenume, media, grupa from studenti where media >= 9) st
   inner join
    (select cod, sectia, anstudiu from grupe where sectia = 2) gr
   on st.grupa = gr.cod
 )
 inner join
  (select cod, denumire from sectii where cod = 2) se
 on gr.sectia = se.cod
 order by anstudiu, nume, prenume
```

Pentru ultima formă a expresiei din algebra relațională se poate construi un arbore de evaluare.



Forma optimala pentru expresia din algebra relationala



Cu informații din dicționarul bazei de date și cu eventuale informații statistice, din ultima formă a expresiei se poate genera un plan de execuție, unde fiecare **operator relațional** se înlocuiește cu un **algoritm de evaluare**.

Bibliografie

- ▶ Leon Tambulea, Curs de Baze de Date, UBB Cluj
- ▶ [Si10] SILBERSCHATZ A., KORTZ H., SUDARSHAN S., *Database System Concepts*, McGraw-Hill, 2010, cap. 13
- ▶ [Si10a] SILBERSCHATZ A., KORTZ H., SUDARSHAN S., <http://codex.cs.yale.edu/avi/dbbook/db6/slide-dir/PDF-dir/ch13.pdf>, 2010
- ▶ [Ga08] GARCIA-MOLINA, H., ULLMAN, J., WIDOM, J., *Database Systems: The Complete Book*, Pearson Prentice Hall, 2008, cap. 16
- ▶ [Da04] DATE, C.J., *An Introduction to Database Systems* (8th Edition), Addison-Wesley, 2004, cap 13
- ▶ [Kn76] KNUTH, D.E., *Tratat de programare a calculatoarelor. Sortare si cautare*. Ed.Tehnica, Bucuresti 1976

